MBR WP Performance

Comprehensive User Guide

A practical walkthrough of every tab, setting, and workflow in the plugin — what each option does, when to use it, and when to leave it alone.

Version	1.9.1
Author	Made by Robert
Website	littlewebshack.com
Support	madebyrobert.co.uk
Updated	April 2026
Licence	GPL v2 or later

Free. No upsells. No premium tiers. No tracking.

Contents

Introduction

MBR WP Performance is an all-in-one WordPress performance optimisation plugin. Rather than hiding everything behind a handful of presets, it exposes each optimisation as an individual toggle with a clear explanation of what it does and when you'd want it on.

Settings are organised across nine tabs, each focused on a specific performance area. The admin UI is accessed from the WordPress toolbar — there's no sidebar menu item, which keeps the wp-admin sidebar tidy. The plugin is completely free with no premium tiers, no upsells, and no external tracking.

Who this guide is for

This guide is aimed at WordPress site owners, developers, and agencies running the plugin on live sites. It assumes familiarity with basic WordPress concepts (plugins, themes, the admin area) but doesn't require deep technical knowledge. Where a particular setting has technical implications, those are called out explicitly in callout boxes.

How to use this guide

Each tab has its own chapter. Within each chapter you'll find a short description of what that area of the plugin targets, followed by a walk-through of each setting with guidance on when to enable it. You don't need to read the guide front-to-back — jump to the tab you're configuring and follow that chapter.

If you're setting up the plugin for the first time, start with the Getting Started chapter which walks you through a safe first-time configuration in about ten minutes.

> **Tip.** Enable one feature at a time and test your site after each change. Performance plugins can interact in surprising ways with themes, page builders, and other plugins. Individual toggles make it easy to narrow down any issues.

Getting Started

Requirements

- WordPress 5.8 or higher
- PHP 7.4 or higher
- MySQL 5.6 or higher
- GD library with WebP support (for image conversion and resizing features)

Installation

Download the latest release zip from littlewebshack.com or the plugin's GitHub releases page, then install it via Plugins → Add New → Upload Plugin. Upload the zip, click Install Now, then Activate. You can also extract the zip and upload the resulting folder to /wp-content/plugins/ via SFTP.

After activation, look for **WP Performance** in the WordPress admin toolbar at the top of the screen. Clicking it takes you straight to the settings page.

A safe first-time setup

If you're just getting started and want a sensible baseline that won't break anything, enable the following settings in this order. Test your site after each step before moving on.

- **Database tab:** Run a post revisions cleanup (keep the last 5), delete expired transients, and run the orphaned metadata scan. This is a housekeeping step that makes everything else faster.
- **Lazy Loading tab:** Enable 'Lazy Load Images' and 'Lazy Load iFrames'. These use native browser support and are extremely safe.
- **Core Features tab:** Enable 'Disable Emojis', 'Disable Embeds' (only if you don't use them), and 'Remove Query Strings'. Skip 'Remove jQuery Migrate' for now.
- **WebP tab:** Check the Server Diagnostics panel first. If it reports GD with WebP support, enable 'Convert on Upload' and run the bulk converter on your existing Media Library.
- **WebP tab → Image Sizing & Dimensions:** Enable 'Resize Large Uploads' (default 2560px) and 'Add Missing Width & Height'. Both directly address common PageSpeed Insights warnings.
- **WooCommerce tab (if applicable):** Disable cart fragments on non-shop pages, enable 'Disable WC Scripts on Non-Shop Pages', and set Action Scheduler retention to 14 days.

> **Note.** The plugin automatically disables all optimisations inside page builder editors (Elementor, Beaver Builder, Divi, Oxygen, Bricks, WPBakery) to prevent conflicts with the editing experience. You don't need to configure this — detection is automatic.

The Admin Interface

The settings page is accessed from the WP Performance item in the admin toolbar. Each tab corresponds to a specific area of optimisation, and all tabs share the same layout: a list of settings with tooltips, a Save Changes button at the bottom, and a Reset to Defaults button.

Tooltips

Every setting has a question-mark icon next to its label. Hovering over it reveals a tooltip with a brief explanation of what the setting does. Tooltips are the fastest way to understand a setting without leaving the admin.

Saving and resetting

The Save Changes button saves the settings on the current tab. Settings on other tabs are not affected. The Reset to Defaults button resets every setting across every tab — use it carefully.

The nine tabs

Tab	Focus area
Core Features	WordPress-level features: emojis, embeds, XML-RPC, REST API, heartbeat
JavaScript	Defer, async, jQuery, minification, delay until interaction
CSS	Critical CSS, async loading, minification, unused styles
Fonts	Google Fonts self-hosting, preloading, font-display, Font Awesome
Preloading	Image preload, fetch priority, speculative loading, early hints
Lazy Loading	Native image and iFrame lazy loading with fine-grained exclusions
Database	Revisions, transients, orphaned metadata, table optimisation
WebP	WebP conversion, bulk converter, image resizing and dimensions
WooCommerce	WooCommerce-specific optimisations (v1.9.0+)

Core Features Tab

The Core Features tab groups WordPress-level toggles that disable default behaviour you may not need. Most of these are safe to enable. A small number require a bit of judgement about what your site actually uses.

Scripts & Styles

Disable Emojis

WordPress injects a JavaScript detection script and inline CSS on every page to support emoji rendering on older browsers. Modern browsers render native emojis without this help. Safe to enable unless you specifically use WordPress's emoji conversion on very old mobile browsers.

Disable Dashicons on Frontend

Dashicons is WordPress's admin icon font. It's loaded on the frontend for logged-in users and sometimes for all visitors depending on the theme. Only enable if you're certain your theme doesn't use Dashicons in its public-facing templates. Check by temporarily enabling it on a staging copy and inspecting your site while logged out.

Disable Embeds

Removes the oEmbed functionality that converts pasted URLs (YouTube, Twitter, etc.) into rich embeds, along with the embed detection script. If you embed external content in posts, leave this off. If your site is purely text and images, it's a useful win.

Remove jQuery Migrate

jQuery Migrate is a compatibility layer that lets older plugins keep working with newer versions of jQuery. Modern, actively maintained plugins rarely need it. Test carefully after enabling — if any plugin breaks, disable this first.

Remove Global Styles

Block themes and Full Site Editing inject a large chunk of inline CSS on every page to handle theme.json design tokens. If you're using a classic (non-block) theme, this CSS is dead weight and can be removed safely.

WordPress Features

Disable XML-RPC

XML-RPC is a remote API that predates the REST API. It's mainly used by the old WordPress mobile apps and Jetpack. If you don't use either, disabling XML-RPC reduces attack surface and blocks a common brute-force vector.

Hide WordPress Version

Removes the WordPress version meta tag and generator strings from the frontend. Security through obscurity only — it won't stop a determined attacker who can fingerprint your site in other ways, but there's no downside to enabling it.

Disable RSS Feeds

If your site doesn't publish content that anyone subscribes to via RSS, this removes the feed URLs and prevents them being scraped. Most brochure sites and e-commerce stores can safely disable feeds. Blogs should leave them on.

Disable Self-Pingbacks

Stops WordPress notifying itself when one post links to another on the same site. Self-pingbacks generate useless comments and extra database activity. Safe to enable.

REST API Control

Three modes are available. 'Default' leaves the REST API fully open as WordPress ships it. 'Logged-in Only' restricts the API to authenticated users, which breaks any frontend feature that uses the API for anonymous visitors — block editor previews, some form plugins, and most headless setups.

'Block Public User Enumeration' is the middle ground: the API stays open for most endpoints but the /users endpoint no longer reveals usernames to unauthenticated requests. This is usually the right choice for brochure sites and e-commerce stores.

Heartbeat API

The Heartbeat API is WordPress's real-time communication layer used by autosave, post locking, and various plugins. It fires an admin-ajax request every 15 seconds by default, which can generate significant server load on busy sites.

Options are 'Default', 'Disable on Dashboard', 'Disable Everywhere', and a custom frequency setting (in seconds). For most sites, setting the frequency to 60 seconds is a good balance between responsiveness and server load.

Post Revisions & Autosave

Controls how many post revisions WordPress stores and how often it autosaves drafts. Reducing revisions to 5 or 10 keeps the posts table lean without losing too much edit history. Autosave every 60 seconds is WordPress's default and rarely needs changing.

Remove Query Strings

Some older caching proxies refuse to cache URLs that contain query strings like ?ver=1.2.3 on CSS and JS files. Removing them improves cacheability. Modern page cache plugins handle this correctly regardless, but the toggle is harmless.

Legacy WooCommerce Toggles

If you were running an older version of the plugin, you may have 'Disable WooCommerce Scripts on Non-Shop Pages' enabled here. From v1.9.0 onwards, all WooCommerce optimisations have moved to their own dedicated tab. The legacy toggles remain here for backward compatibility — your settings keep working — but the newer WooCommerce tab offers more granular control and new features. See the WooCommerce chapter.

JavaScript Tab

The JavaScript tab controls how and when JS files load on your site. Used thoughtfully, the options here can produce large improvements in Time to Interactive and First Contentful Paint scores. Used carelessly, they can break your site. Test after every change.

Defer JavaScript

Adds the 'defer' attribute to script tags, which tells the browser to download the script in parallel with HTML parsing but only execute it after the HTML is fully parsed. This prevents render-blocking but preserves execution order.

Almost all scripts can safely use defer. The main exceptions are scripts that the theme relies on for inline execution during page render — but those are increasingly rare.

Async JavaScript

Adds the 'async' attribute, which tells the browser to download in parallel and execute as soon as the download completes — regardless of DOM readiness. This is more aggressive than defer and can reorder script execution.

Only use async for self-contained scripts that don't depend on other scripts or the DOM being in a specific state. Analytics beacons are classic candidates; anything that uses jQuery is almost certainly not.

Move Scripts to Footer

Hoists <script> tags out of the <head> and into the footer. Render-blocking JS in the head delays the first paint; moving it to the footer means the user sees content sooner. Combine with defer for the biggest effect.

Remove jQuery

> **Warning.** This is the most disruptive toggle in the plugin. An enormous amount of WordPress code — themes, plugins, page builders — depends on jQuery being available globally. Enable only if you know your theme and every active plugin ships without jQuery dependencies. Test extensively on a staging copy first.

Minify JavaScript

Strips whitespace and shortens variable names in served JS files, reducing file size at the cost of some CPU time during delivery. Compatible with most themes and plugins. If you see a JS error immediately after enabling, disable and report which script broke — some poorly written scripts can't be minified safely.

Combine JavaScript

Merges multiple JS files into a single bundle, reducing the number of HTTP requests. On HTTP/2 servers, combining has diminished returns because modern browsers multiplex requests efficiently. On older HTTP/1.1 hosts, it remains useful. Compatibility issues are more common than with minification.

Delay JavaScript Until Interaction

This is a powerful setting. Any script matching the delay list won't execute until the user scrolls, clicks, hovers, or presses a key. This can push analytics, chat widgets, and tag managers entirely off the critical rendering path.

Common candidates for delay: Google Tag Manager, Google Analytics, Facebook Pixel, Hotjar, Intercom, Tawk.to, HubSpot tracking. Use the exclusion list to prevent delays on scripts that must run immediately (the Cookie Consent plugin's own script, for instance).

> **Tip.** Delay until interaction can dramatically improve Lighthouse scores, but measure the impact on conversion data. Analytics scripts that delay will miss users who bounce before interacting. Some analytics providers accept this trade; others don't.

Exclusion Lists

Each of the above options (defer, async, delay) has its own exclusion list. Paste one script identifier per line — you can match on filename, handle, path fragment, or inline script content. Exclusions are applied as a simple substring match, so being too loose (say, 'js') will exclude everything.

CSS Tab

The CSS tab manages stylesheet delivery. The flagship feature here is critical CSS generation, which isolates the above-the-fold styles and inlines them in the document head so the first paint happens without waiting for full stylesheets to download.

Minify CSS

Strips whitespace and comments from served CSS. Almost always safe. The savings per file are small, but they add up on sites with many stylesheets.

Combine CSS

Merges multiple stylesheets into one bundle. Same HTTP/2 caveat as the JavaScript combine option — the benefit has reduced on modern hosts. Watch for cascade ordering issues: if your theme relies on specific stylesheet load order, combining can break styling.

Load CSS Asynchronously

Changes CSS loading from render-blocking to non-blocking. Without critical CSS, this produces a Flash of Unstyled Content (FOUC) — the page renders briefly with no styles before the CSS arrives. Always pair this with the critical CSS generator.

Critical CSS Generator

The critical CSS generator scans your homepage and extracts the styles needed for the initial viewport. It then saves that subset as inline CSS which is served in the head, letting the full stylesheet load asynchronously without FOUC.

Click 'Generate Critical CSS' and wait for the scan to complete. The generated CSS is stored as plain text which you can manually edit if needed. Regenerate after major design changes.

> **Note.** The built-in critical CSS generator covers your homepage. Inner pages with significantly different layouts (product pages, long-form posts, landing pages) may need per-template critical CSS, which isn't currently automated. For most sites the homepage critical CSS is close enough for other pages to avoid obvious FOUC.

CSS Scanner (Unused Styles)

The scanner analyses which CSS rules from your stylesheets are actually used on the homepage. It won't automatically remove unused CSS — that's too risky — but it gives you a report showing which files have the most unused styles, useful for deciding where to focus manual optimisation.

Disable Global Styles

Duplicate of the Core Features toggle, placed here for discoverability. Only enable one or the other.

Separate Block Styles

Tells WordPress to only load the CSS for blocks that actually appear on the page, rather than loading the full block library stylesheet on every page. Pure win on sites that use the block editor.

Disable Elementor Google Fonts

Elementor injects Google Fonts link tags based on its widget configuration. If you've self-hosted your fonts via the Fonts tab, this toggle stops the duplicate external requests. Only relevant on Elementor sites.

Legacy WooCommerce CSS Toggle

Like the Core Features tab, this retains the old 'Disable WooCommerce CSS on Non-Shop Pages' option for backward compatibility. New installs should use the WooCommerce tab for this instead.

Fonts Tab

Web fonts are one of the most commonly mishandled performance costs on WordPress sites. Every Google Fonts request introduces render-blocking HTTP, DNS resolution, and TLS handshakes against third-party domains. The Fonts tab gives you tools to bring fonts under your own domain's control.

Self-Host Google Fonts

The headline feature. Enable 'Self-Host Google Fonts' and the plugin scans your site for Google Fonts requests, downloads the font files to your server, and rewrites the stylesheet URLs to serve from your own domain. No more calls to fonts.googleapis.com or fonts.gstatic.com.

Click 'Download Fonts' to perform the initial scan and download. After changing your font stack (in the theme or page builder), click 'Clear Font Cache' and re-download.

> **Note.** Self-hosting Google Fonts also addresses GDPR concerns in the EU. When Google Fonts are served from Google's CDN, visitor IP addresses are transmitted to Google. Self-hosting keeps that data on your server.

Manual Fonts

If your theme declares Google Fonts in a way the automatic scanner doesn't detect — for example, hard-coded in a PHP template — you can list the font families manually in this field. The plugin downloads them on save. Useful for edge cases where detection fails.

Preload Critical Fonts

Preloading fonts tells the browser to start downloading them as soon as possible, rather than waiting until the CSS that uses them is parsed. Enable 'Preload Critical Fonts' and list the fonts that appear above the fold. Don't preload every font — it hurts more than it helps.

Font Display Strategy

Controls what the browser does while a web font is still loading. Four options:

- **swap:** Show the fallback font immediately, then swap to the web font when it loads. Best for performance; there will be a visible text swap.
- **block:** Hide text for up to 3 seconds waiting for the font, then fall back. Rarely a good choice — invisible text looks broken.
- **fallback:** Short block period then fallback; if the font arrives quickly, use it, otherwise keep the fallback for the rest of the page.
- **optional:** Like fallback but even more aggressive — if the font is slow, it won't be used at all on the current page load. Best font-display for flaky connections.

Default to 'swap' unless you have a specific reason to choose otherwise.

Font Subsetting

When enabled, the plugin strips glyphs from downloaded fonts that aren't needed for the character sets you've selected. A Latin-only subset can be 60-80% smaller than the full file. Safe for English and Western European sites; be careful if your content includes Cyrillic, Greek, or CJK characters.

Font Awesome Optimisation

Font Awesome often arrives as a second, large web font on sites that already bundle icons via their theme. Options are 'Default' (no change), 'Optimise' (load only the subset actually used), and 'Disable' (don't load Font Awesome at all). Use 'Disable' only if you're certain nothing on your site uses Font Awesome icons.

Preloading Tab

Preloading and speculative loading are hints you give the browser about what resources it will need soon. Used judiciously, they reduce perceived latency. Overused, they waste bandwidth and can actually hurt performance. The options here reward careful thought.

Preload Critical Images

Tells the browser to start downloading specific images as soon as it parses the HTML, rather than waiting until they're encountered in the DOM. The most common use case is the Largest Contentful Paint (LCP) image — the hero image at the top of a landing page.

List image URLs or CSS selectors to identify which images should be preloaded. Be specific: preloading images that are below the fold wastes bandwidth and competes with resources the browser actually needs.

Fetch Priority

The 'fetchpriority' HTML attribute lets you explicitly tell the browser which images matter most. Three related settings:

- **Auto-prioritise First Image:** Automatically sets fetchpriority='high' on the first image found in the main content area. Sensible default for most sites.
- **High Priority Selectors:** CSS selectors for additional critical images that should get high priority. Good for landing pages with multiple above-the-fold images.
- **Disable Core Fetch Priority:** WordPress core added its own fetchpriority logic in 6.3, which sometimes conflicts with manual optimisation. Disable if you want complete control.

Cloudflare Early Hints

HTTP 103 Early Hints let your server tell Cloudflare (or another edge cache) which resources to start preloading even before the origin finishes generating the HTML response. If your site is behind Cloudflare with Early Hints enabled in the dashboard, enabling this toggle tells the plugin to emit the Link headers Cloudflare needs.

Speculative Loading

The Speculation Rules API lets browsers prefetch or prerender the next page the user is likely to visit. When they click a link that's been pre-rendered, the page appears instantly. Four eagerness settings:

- **Conservative:** Only prefetch on explicit mouseover or touchstart.
- **Moderate:** Default. Prefetch links on hover.
- **Eager:** Start prefetching as soon as a link appears in the viewport.
- **Auto:** Let the browser decide.

> **Warning.** Speculative loading consumes server resources even for pages the user never ends up visiting. On high-traffic sites, 'Eager' can significantly increase server load. Start with 'Moderate' and monitor your server metrics before escalating.

Lazy Loading Tab

Lazy loading delays the loading of images and iFrames until they're about to scroll into view. The plugin uses the browser's native loading='lazy' attribute where possible, falling back to IntersectionObserver for older browsers and for features native lazy loading doesn't support.

Lazy Load Images

Enables lazy loading for all <img> tags on the frontend. Native lazy loading is supported in all modern browsers and produces essentially no CLS when image dimensions are known — pair it with the 'Add Missing Width & Height' option in the WebP tab.

Lazy Load iFrames

Same treatment for embedded iFrames (YouTube, Vimeo, Google Maps, Twitter embeds). Often a bigger win than image lazy loading because iFrames bring their own JS payloads. Enable this early.

Lazy Load Background Images

CSS background images (specified via 'background-image: url(...)' in a stylesheet) aren't covered by native lazy loading. This option uses IntersectionObserver to apply the styles only when the container scrolls into view. Requires adding a specific class or data attribute to the containers — check the tooltip for the exact markup.

Lazy Threshold

How far outside the viewport an image can be before the browser starts loading it. The default (500 pixels below the fold) strikes a balance between bandwidth savings and avoiding blank spots when the user scrolls quickly. Reduce for mobile-heavy audiences on slow connections; increase for sites where users scroll fast.

Fade-in Animation

When enabled, lazy-loaded images fade in over a configurable duration rather than popping in abruptly. Cosmetic but pleasant. Add about 200ms of duration for a subtle effect without feeling slow.

Exclusion Options

Fine-grained controls for excluding specific images from lazy loading. Exclude by:

- **CSS selector:** Any valid selector, e.g. '.hero-image' or '#logo'
- **Class name:** Exclude by class (without the leading dot)
- **ID:** Exclude by element ID (without the hash)
- **Data attribute:** Exclude any element with a matching data attribute
- **Keyword in filename:** Substring match on the image URL
- **Parent container:** Exclude all images within a matching parent

> **Tip.** Always exclude your LCP image from lazy loading. Lazy loading the Largest Contentful Paint image defeats the purpose — the whole point is that image should appear as early as possible.

DOM Monitoring

Images that are added to the page dynamically (by a carousel, infinite scroll, AJAX-loaded content, or a page builder's frontend editing) aren't covered by a one-shot scan. DOM monitoring watches for new images being added and applies lazy loading to them too. Small performance cost on pages with lots of DOM mutations.

Database Tab

The Database tab handles housekeeping: pruning accumulated bloat, optimising tables, and keeping storage lean. Regular maintenance here keeps queries fast as your site ages.

> **Warning.** Always take a full database backup before running any of the cleanup operations in this tab. The operations are permanent and cannot be undone. This is your first and last line of defence against mistakes.

Post Revisions

WordPress saves a copy of every draft as a post revision, forever by default. On busy editorial sites, this can double or triple the size of the wp_posts table. Two settings:

- **Keep Last X Revisions:** Limits the number of revisions stored per post going forward. 5 is usually enough.
- **Delete Existing Revisions:** A one-click button that prunes existing revisions to match the 'Keep Last X' limit.

Auto-Drafts & Trash

WordPress creates an auto-draft every time you click 'Add New'. Most of these are abandoned and never cleaned up. Similarly, trashed posts stay in the trash forever unless you explicitly empty it. Enable the auto-cleanup schedules to keep these table sections from growing.

Spam & Unapproved Comments

Comments marked as spam, trash, or unapproved linger in the database. If you review and clear them manually, these toggles aren't necessary. If you rely on Akismet to silently move spam to a pile and forget about it, enable automatic cleanup on a schedule (weekly is fine).

Orphaned Metadata Scanners

Four separate scanners find metadata that references records which no longer exist:

- **Orphaned Post Meta:** wp_postmeta rows pointing to deleted posts
- **Orphaned Comment Meta:** wp_commentmeta rows pointing to deleted comments
- **Orphaned Taxonomy Relationships:** wp_term_relationships rows pointing to deleted objects
- **Orphaned Term Meta:** wp_termmeta rows pointing to deleted terms

Each scanner has a two-step workflow: Scan (count), then Delete (remove). Scanning is always safe. Deleting is permanent but usually very safe because the referenced records are already gone.

Transients

Transients are short-lived cached data in the wp_options table. Expired transients should be cleaned by WordPress automatically but often aren't, and poorly written plugins can leave hundreds of thousands of them behind.

- **Get Transient Stats:** See how many transients exist, how many are expired, and total storage size.

- **Delete Expired Transients:** Removes only transients whose expiration has passed. Completely safe.
- **Delete All Transients:** Nuclear option. Removes every transient including active ones. Plugins that depend on transients will re-generate them on next use.

Table Operations

Three database-level maintenance operations:

- **Optimise Tables:** Runs OPTIMIZE TABLE on every table. Reclaims space from deleted rows and defragments the table files. Safe but can be slow on large databases.
- **Convert MyISAM to InnoDB:** If any of your tables are still using the legacy MyISAM engine, this converts them to InnoDB. InnoDB offers row-level locking, transactions, and crash recovery.
- **Repair Tables:** Runs REPAIR TABLE on corrupted tables. Only relevant if you're actually seeing 'table marked as crashed' errors.

Database Info

A diagnostic panel showing table sizes, row counts, engine types, and character sets. Useful for identifying which tables have grown unexpectedly large.

WebP Tab

The WebP tab handles two related but distinct jobs: converting your existing images to the WebP format for bandwidth savings, and managing image sizing and dimensions to fix common PageSpeed Insights warnings. The tab is split into two major sections.

Server Diagnostics

Before enabling any conversion, check the Server Diagnostics panel. It reports:

- GD library version (should be 2.0 or higher)
- WebP encoder support (must be present)
- Upload folder write permissions (the converter writes WebP files next to originals)
- .htaccess writability (only relevant if you plan to use the rewrite rules option)

If WebP support is missing, ask your host to rebuild PHP's GD library with libwebp. Most reputable hosts support this.

Conversion Settings

Convert on Upload

When enabled, every new image uploaded to the Media Library is automatically converted to WebP in the background. Doesn't touch the original JPG or PNG — the WebP is created alongside it.

Compression Level

WebP quality from 1 (tiny, blurry) to 100 (identical to original). The default of 80 is a good compromise. Sites with a lot of photography may want 85-90; sites that prioritise speed above all can push down to 70.

Delivery Method

Two options for serving WebP to supporting browsers:

- **Picture tag:** The plugin rewrites <img> tags as <picture> elements with both WebP and original sources. Works on any hosting environment. Introduces a small amount of DOM overhead.
- **.htaccess rewrite:** Apache and LiteSpeed hosts can use URL rewriting to transparently serve WebP when the browser accepts it. Cleaner markup but depends on a specific server environment.

Bulk Converter

To convert your existing Media Library, scroll to the Bulk Converter section. Click Start Conversion and the plugin iterates through every image, converting to WebP with your configured compression level.

The converter shows live progress, skip reasons (usually 'WebP would be larger than original'), and a running savings total. You can pause and resume at any time. On Media Libraries with thousands of images, expect the process to run for a while — let it complete in one sitting if possible.

Conversion History

Every WebP file the plugin creates is recorded in its registry. The Conversion History section shows what's been converted, with options to:

- **Delete selected WebPs:** Remove specific WebP files, freeing disk space. Originals are untouched.
- **Revert All:** Delete every plugin-created WebP file. Clean slate. Originals are still untouched.

> **Note.** The plugin never modifies, moves, or deletes your original images. Every WebP is a parallel file. 'Revert All' only removes the WebP files — your Media Library contents are always preserved.

Image Sizing & Dimensions

This section targets two specific PageSpeed Insights warnings. The settings are independent from the WebP conversion above — you can use one without the other.

Resize Large Uploads

Any image uploaded larger than the configured maximum dimension (default 2560px on the longest edge) is automatically scaled down on upload using the WordPress core pipeline. Aspect ratio is preserved. The original file is never stored — the scaled version replaces it.

Add Missing Width & Height

Scans frontend HTML for <img> tags missing width or height attributes and adds them based on the actual image dimensions. Giving the browser the aspect ratio up front prevents Cumulative Layout Shift. Works with post content, Gutenberg blocks, Elementor widgets, attachment images, and post thumbnails.

Bulk Resize Tool

The 'Resize Large Uploads' setting only affects images uploaded in the future. To downscale images that already exist in your Media Library, use the Bulk Resize tool.

> **Warning.** The Bulk Resize tool permanently overwrites files on disk. Original files are not preserved. Always take a full file-system and database backup before running it. This is the most destructive tool in the plugin.

The workflow is two-phase. Click Scan to see which images exceed the threshold. Review the list. Click Start Resize to begin. A progress bar, live log, and running savings total show what's happening. After resize, the plugin regenerates all registered sub-sizes using the WordPress core pipeline and cleans up any stale plugin-created WebP files so they'll be regenerated from the new dimensions.

WooCommerce Tab

Added in v1.9.0. The WooCommerce tab only activates when WooCommerce is installed and active. It consolidates the WooCommerce-specific optimisations that used to be scattered across the Core and CSS tabs, and adds a set of deeper controls that target WooCommerce's most common performance drains.

> **Note.** If you previously had 'Disable WooCommerce Scripts on Non-Shop Pages' enabled on the Core tab, or 'Disable WooCommerce Styles on Non-Shop Pages' on the CSS tab, those settings continue to work unchanged. The new WooCommerce tab offers equivalent and additional options. You can migrate when convenient.

Geolocation Advisory

At the top of the tab, the plugin checks your WooCommerce geolocation setting and displays a notice if it's configured in a way that interacts badly with full-page caching. Two problematic modes:

- **Geolocate:** WooCommerce performs a server-side IP lookup on every page load. Full-page caches are effectively disabled because every response varies per visitor.
- **Geolocate (with page cache support):** WooCommerce appends a ?v=<timestamp> query string to URLs. Depending on your cache plugin's configuration, this can either bypass the cache entirely or create a very large number of cache variants.

The advisory notice includes a direct link to WooCommerce's General settings where you can change the setting to 'Shop base address' (no geolocation), which is cache-friendly.

Frontend Assets

Cart Fragments

WooCommerce fires an admin-ajax request called wc-cart-fragments on every single page load to update the mini-cart display. On cached sites, this uncached request is often the single biggest source of Time to First Byte latency. Three options:

- **Default (WooCommerce behaviour):** Fragments run on every page. Mini-cart updates live.
- **Disable on non-shop pages:** Fragments run on shop, cart, checkout, product, and account pages only. Mini-cart may be stale elsewhere until page reload.
- **Disable site-wide:** Fragments never run. Mini-cart updates only on page reload. Biggest performance win.

> **Tip.** Most stores can safely disable cart fragments site-wide. The trade-off is that if your theme has a persistent header with a cart count, the count won't update live after an 'Add to Cart' — the user has to refresh to see the new count. If that's acceptable, the performance win is substantial.

Disable WC Scripts on Non-Shop Pages

Expanded version of the old Core tab toggle. Dequeues WooCommerce itself, wc-add-to-cart, wc-cart, wc-checkout, selectWoo, jquery-blockui, js-cookie, and the select2 library on any page that isn't a shop, product, cart, checkout, or account page.

Disable WC Styles on Non-Shop Pages

Equivalent for stylesheets. Removes woocommerce-general, woocommerce-layout, woocommerce-smallscreen, and related stylesheets from non-shop pages.

Disable WC Block Assets on Non-Shop Pages

WooCommerce's block editor assets (wc-blocks-style, wc-blocks-vendors-style, and their supporting scripts) load on many themes even when no blocks are in use. Safe to disable on non-shop pages if you're not using the Cart or Checkout blocks on landing pages.

Disable Password Strength Meter

WooCommerce includes the zxcvbn password strength library for the account registration flow. It's loaded on every frontend page in many themes. Disabling saves around 50KB on non-registration pages.

Admin Options

Disable Marketplace Suggestions

Stops WooCommerce fetching extension recommendations from WooCommerce.com and showing them inside your admin. Removes outbound API calls and declutters the admin experience.

Disable Dashboard Widgets

Removes the WooCommerce Status, Recent Reviews, and setup widgets from the WordPress dashboard. Fewer database queries on dashboard load.

Disable WC Admin Scripts on Non-WC Pages

WooCommerce loads its heavy wc-admin React bundles on many non-WooCommerce admin screens. Enabling this dequeues them where they're not needed. Makes the regular wp-admin noticeably snappier on stores that also use heavy-admin plugins like Yoast or ACF.

Database Cleanup

Enable Weekly Automated Cleanup

Added in v1.9.1. When enabled, the plugin's built-in weekly cron runs three maintenance jobs automatically: expired sessions, WooCommerce transients, and Action Scheduler history cleanup. The tab shows the next scheduled run time and a last-run log with what was cleared.

Action Scheduler Retention

WooCommerce's Action Scheduler (the background job queue) keeps completed and failed actions in the wp_actionscheduler_actions table for 30 days by default. On busy stores — especially those running WooCommerce Subscriptions — this table can grow to hundreds of thousands of rows, slowing admin queries noticeably. Options for 14, 7, and 3 day retention.

The retention filter only affects future cleanup runs. To drain an existing backlog, use the 'Run Cleanup Now' button next to the retention setting.

Expired Sessions

Shows a count of expired rows in the wp_woocommerce_sessions table and a button to delete them. WooCommerce's own cleanup cron often fails on sites with flaky WP-Cron, so this is a useful manual lever.

WooCommerce Transients

One-click cleanup of product, shop order, and expired transients using WooCommerce's own helper functions. These transients accumulate from everyday shop operations and clearing them occasionally keeps the options table lean.

Multisite Network Support

The plugin has full WordPress Multisite compatibility from v1.5.0 onwards. Network-activate it and manage default settings from the Network Admin, with per-site override control for situations where individual sites need something different.

Network Admin settings

Go to Network Admin → Settings → WP Performance. You'll see the same tab structure as the regular admin, but the settings you save here become the network defaults for every site.

Push to all sites

After configuring your network defaults, click 'Push Settings to All Sites' to copy them over to every site in the network. You can also select specific sites. Existing site-level settings are overwritten by this operation.

Import from a site

Already have one site configured the way you want? Import its settings as the network default with a single click. Useful when migrating from per-site configuration to network-wide consistency.

Per-site override control

A master toggle in the Network Admin determines whether individual site admins can change their settings or whether they're locked to the network defaults. Super admins can always change settings regardless of this toggle.

New sites

Sites created after the plugin has been network-activated automatically inherit the network defaults on creation. No manual setup per site.

Common Workflows

A set of recipes for common goals. Each assumes you've taken backups and are testing changes on a staging environment first.

"I want to improve my Google PageSpeed Insights score"

- **Lazy Loading tab:** Enable image and iFrame lazy loading. Set an exclusion for your LCP image (usually the hero).
- **Preloading tab:** Enable 'Auto-prioritise First Image'. Add your LCP image URL to 'Preload Critical Images'.
- **CSS tab:** Enable 'Minify CSS', 'Load CSS Asynchronously', and click 'Generate Critical CSS'.
- **JavaScript tab:** Enable 'Defer JavaScript' and 'Move Scripts to Footer'. Add analytics scripts to the 'Delay Until Interaction' list.
- **Fonts tab:** Enable 'Self-Host Google Fonts' and click 'Download Fonts'. Set font-display to 'swap'.
- **WebP tab:** Run the bulk WebP converter. Enable 'Resize Large Uploads' and 'Add Missing Width & Height'.

"My WooCommerce store feels sluggish"

- **WooCommerce tab:** Set cart fragments to 'Disable site-wide' (or 'non-shop pages' if you rely on the live mini-cart).
- Enable 'Disable WC Scripts on Non-Shop Pages' and 'Disable WC Styles on Non-Shop Pages'.
- Enable 'Disable WC Admin Scripts on Non-WC Pages' to speed up the regular wp-admin.
- Set Action Scheduler retention to 14 or 7 days.
- Enable 'Weekly Automated Cleanup' and click 'Run Cleanup Now' once to drain any backlog.
- Check the geolocation advisory — if it's shown, switch WooCommerce to 'Shop base address'.

"I need to meet GDPR requirements on font delivery"

- **Fonts tab:** Enable 'Self-Host Google Fonts' and click 'Download Fonts'. This stops sending visitor IPs to Google's font servers.
- Verify in your browser's Network tab that no requests go to fonts.googleapis.com or fonts.gstatic.com after the change.

"My site has ballooned and I want to clean up the database"

- **Database tab:** Take a full backup first. Always.
- Set 'Keep Last X Revisions' to 5 and click 'Delete Existing Revisions'.
- Run all four orphaned metadata scanners. Delete the results from each.
- Click 'Get Transient Stats' then 'Delete Expired Transients'.
- Click 'Optimise Tables'.
- Check 'Database Info' before and after to see the savings.

"I want to delay analytics without breaking them"

- **JavaScript tab:** Enable 'Delay JavaScript Until Interaction'.
- Add these identifiers to the delay list (one per line): google-analytics, gtag, googletagmanager, fbq, fbevents, hotjar, clarity, intercom.
- Test your site — scripts should only fire after first scroll, click, or keypress.
- Verify in your analytics dashboard that events still record (with a small delay).

Troubleshooting

A setting broke my site

The fastest recovery: open a private browsing window, log into wp-admin, open the plugin settings, and disable the setting you most recently enabled. Save. If you can't access wp-admin, rename the plugin folder via SFTP — that deactivates the plugin entirely. Settings are preserved for when you reactivate.

After enabling 'Remove jQuery', something broke

jQuery is a sprawling dependency. Disable 'Remove jQuery' first and test whether the site recovers. If it does, your theme or a plugin depends on jQuery — don't enable the option. There's no reliable shortlist of what works and what doesn't.

Fonts look different after enabling self-host

The most common cause is that the initial scan missed a font that's declared somewhere unusual. Click 'Clear Font Cache' in the Fonts tab, then add the missing font families to the 'Manual Fonts' field and click 'Download Manual Fonts'. The manual list is a safety valve for cases the scanner doesn't catch.

PageSpeed Insights still reports 'Image elements do not have explicit width and height'

Ensure 'Add Missing Width & Height' is enabled in the WebP tab. Clear any page caches. The feature works at runtime on the frontend HTML, so cached versions of pages will still show the old markup until the cache is flushed.

Critical CSS was generated but I see FOUC

Some themes inject styles in ways the scanner can't reliably capture. Try regenerating critical CSS after you've made any recent design changes. If FOUC persists, manually edit the generated critical CSS via the textarea in the CSS tab to add any missing rules. Regenerate after editing to keep your changes.

WooCommerce geolocation notice keeps appearing

The notice only appears when WooCommerce's default customer location is set to 'Geolocate' or 'Geolocate (with page cache support)'. Go to WooCommerce → Settings → General and change 'Default customer location' to 'Shop base address'. Save. The notice will disappear on the next page load.

The weekly cleanup doesn't appear to be running

WordPress's cron is not a real cron — it fires when someone visits the site. On low-traffic sites, scheduled events can miss their window. Check the 'Next scheduled run' line in the WooCommerce tab; if it's in the past, your server-side cron is unhealthy. Two fixes: either set up a real cron job calling wp-cron.php every 5 minutes, or use WP-CLI to trigger manually with 'wp cron event run mbr_wp_performance_database_cleanup'.

The plugin settings page won't load

Very rare, but usually caused by a conflict with another performance plugin. Deactivate other performance plugins temporarily. If the settings page loads after deactivation, the conflict is likely with rewriting or script handling. Re-enable them one at a time to identify which plugin is the culprit.

Frequently Asked Questions

Will this plugin break my site?

It's designed to be safe, but performance optimisations always carry some risk because they interact with themes and other plugins in complex ways. Always take a backup, test on staging, and enable features one at a time. Features are independently toggleable so you can always reverse a specific change.

Can I use this alongside a caching plugin?

Yes. The plugin provides complementary optimisations that work with any caching solution — WP Rocket, W3 Total Cache, LiteSpeed Cache, WP Super Cache, and others. There's deliberate no overlap: this plugin doesn't implement full-page caching and doesn't try to replace your cache layer.

Is this plugin GDPR compliant?

The plugin itself doesn't collect, transmit, or store any visitor data. Features like self-hosted Google Fonts actively improve GDPR compliance by preventing visitor data from being sent to Google. No tracking, no phoning home, no external calls.

Does the WebP converter modify my original images?

No. WebP files are always created alongside the originals with the same filename and a .webp extension. The originals are never touched. Deactivating the plugin or clicking 'Revert All' removes the WebP files; the originals remain.

What happens if I deactivate the plugin?

All optimisations are immediately disabled. Settings are preserved in case you reactivate later. WebP files tracked in the registry are cleaned up automatically. Note that the Bulk Resize tool is destructive — resized files are permanent and are not restored on deactivation, so always take a backup before using it.

Is WebP conversion the same as image sizing?

No. They're complementary. WebP conversion creates a second copy of each image in the more efficient WebP format and serves that to supporting browsers, without changing dimensions. Image sizing changes the actual pixel dimensions so the browser doesn't download files larger than it needs to display. Use both for the biggest win.

Does the plugin work with Elementor / Beaver Builder / Divi / Oxygen / Bricks?

Yes. The plugin automatically disables all optimisations inside page builder editors to prevent conflicts with the editing experience. On the frontend, all optimisations apply normally. No configuration needed — detection is automatic.

Does the plugin slow down the wp-admin?

No — most optimisations apply only to the frontend. The admin-side features (like 'Disable WC Admin Scripts on Non-WC Pages' in v1.9.0+) actively make wp-admin faster.

How do I update the plugin?

Download the latest zip from littlewebshack.com, then in wp-admin go to Plugins → Add New → Upload Plugin. Upload the new zip and WordPress will offer to replace the existing installation. Settings are preserved through updates.

Where can I report bugs or request features?

GitHub Issues at github.com/harbourbob/mbr-wp-performance is the fastest route. You can also email via madebyrobert.co.uk or leave a message via littlewebshack.com.

Is there a Pro version?

No. There are no premium tiers, no paid upgrades, and no feature gating. Everything in the plugin is available for free. If you find it useful, a contribution via buymeacoffee.com/robertpalmer is always appreciated.

Resources & Support

Where to get help

- **Website:** littlewebshack.com — plugin documentation, downloads, blog posts
- **Author:** madebyrobert.co.uk — agency site and contact form
- **GitHub:** github.com/harbourbob/mbr-wp-performance — issues and feature requests
- **Buy me a coffee:** buymeacoffee.com/robertpalmer — tip jar if the plugin has saved you time

Related plugins in the portfolio

MBR WP Performance is one of a family of free plugins from Made by Robert. Others worth knowing about:

- **MBR Advanced Asset Manager:** Conditional dequeuing of scripts and styles on specific URLs — complements this plugin's more blanket optimisations
- **MBR Cookie Consent:** Privacy-first cookie consent banner with IAB TCF v2.3 and Google Consent Mode v2 support
- **MBR WP Site Detector:** Scans URLs to detect whether they're running WordPress
- **MBR AI Chatbot:** Drop-in AI chat widget with Claude and ChatGPT support

Philosophy

Every plugin in the Made by Robert portfolio is free, with no premium tiers, no upsells, and no tracking. The goal is to make the kind of tools site owners actually need available without friction. If you find the plugin useful, consider leaving a thumbs-up on its GitHub repository or sharing it with someone who might benefit.

Thanks for using MBR WP Performance. Happy optimising.